```
function
ps_defo_bowl(ps_data,Nifgs,Npsc,Npsp,Btemp,std_param,xc0,yc0,zc0,r0,r10,epoch,de
fo_method,z)


% defo bowl
%
% Input:  - ps_data     input data
%         - Nifgs       number of interferograms
%         - Npsc        number of psc
%         - Npsp        number of psp
%         - Btemp       temporal baselines
%         - std_param   standard deviations for pseudo-
%                       observations
%         - xc0         initial origin azimuth coordinate of the subsidence bowl
%         - yc0         initial origin range coordinate of the subsidence bowl
%         - zc0         initial origin z coordinate of the subsidence bowl
%         - r0          initial radius of subsidence bowl (1 per epoch)
%         - r10         initial depth of subsidence bowl (1 per epoch)
%         - epoch       indices of epochs
%         - defo_method 1 = r1, r, xc, yc, zc
%                       2 = r1, r, zc
%                       3 = r1, zc
%         - z           psc selection
%
% ------------------------------------------------------------------------
% File.............: ps_defo_bowl.m
% Version & Date..: 1.7.2.16, 12-DEC-2009
% Authors.........: Astrid Humme
%                   Freek van Leijen
%                   Delft Institute of Earth Observation and Space Systems
%                   Delft University of Technology
% ------------------------------------------------------------------------
%
% This software is developed by Delft University of Technology and is
% intended for scientific use only. Applications for commercial use are
% prohibited.
%
% Copyright (c) 2004-2009 Delft University of Technology, The Netherlands
%
% Change log
%


% ------------------------------------------------------------------------
% Initialize
% ------------------------------------------------------------------------

global project_id Npar_max Nlines Npixels az_spacing r_spacing m2ph fig
global ps_eval_method max_mem_buffer

Nps = size(ps_data,1);

if isempty(epoch), epoch = 1:Nifgs; end
Nepoch = length(epoch);

%index = find(abs(Btemp(epoch))<0.1); %remove small Btemp (singular Amatrix)
%if ~isempty(index), epoch(index) = []; Nepoch = length(epoch); end

if isempty(xc0), xc0 = az_spacing*Nlines/2; end
if isempty(yc0), yc0 = r_spacing*Npixels/2; end
if isempty(zc0), zc0 = 0; end
if isempty(r0), r0 = repmat((az_spacing*Nlines+r_spacing*Npixels)/16,Nepoch,1);
end
```

```
if isempty(r10), r10 = std_param(4)*Btemp(epoch); end

[Nrow,Ncol] = size(r0);
if Nrow==1, r0 = r0'; end
[Nrow,Ncol] = size(r10);
if Nrow==1, r10 = r10'; end

xc = xc0;
yc = yc0;
zc = zc0;
r = r0;
r1 = kron(r10,ones(Nps,1));;

ps_az = ps_data(:,2)*az_spacing;
ps_r = ps_data(:,3)*r_spacing;

y = ps_data(:,Npar_max+7:Npar_max+6+Nifgs);
y = y(:,epoch);
y = reshape(y,Nps*Nepoch,1);

% ------------------------------------------------------------------------
% Start iteration
% ------------------------------------------------------------------------

fprintf(1,'\n');
fprintf(1,'Iteration has started....\n');

v = 1;
q = inf;
wtest_flag = 'no';
while q > 1e-10

  fprintf(1,'\n');
  fprintf(1,'Iteration no %g.....\n',v);

  q_old = q;

  % ------------------------------------------------------------------------
  % Setup functional model
  % ------------------------------------------------------------------------

  C = sparse(-exp(-0.5*kron(r.^-2,(ps_az-xc).^2 + (yc-ps_r).^2)));

  dr1 = sparse(Nps*Nepoch,Nepoch);
  for k = 1:Nepoch
    dr1(((k-1)*Nps+1):Nps*k ,k) = C(((k-1)*Nps+1):Nps*k,1);
  end

  switch defo_method
    case 1
      drr = r1.*C.*kron((r.^-3),((ps_az-xc).^2+(yc-ps_r).^2));
      dr = sparse(Nps*Nepoch,Nepoch);
      for k = 1:Nepoch
        dr(((k-1)*Nps+1):Nps*k ,k) = drr(((k-1)*Nps+1):Nps*k,1);
      end

      dxc = sparse(r1.*C.*kron(r.^-2,ps_az-xc));

      dyc = sparse(r1.*C.*kron(r.^-2,ps_r-yc));

      dzc = sparse(repmat(1,Nps*Nepoch,1));

      dxA = [dr1 dr dxc dyc dzc];
      clear dxc dyc dzc dr dr1 drr k
```

```
  case 2
    drr = r1.*C.*kron((r.^-3),((ps_az-xc).^2+(yc-ps_r).^2));
    dr = sparse(Nps*Nepoch,Nepoch);
    for k = 1:Nepoch
      dr(((k-1)*Nps+1):Nps*k ,k) = drr(((k-1)*Nps+1):Nps*k,1);
    end

    dzc = sparse(repmat(1,Nps*Nepoch,1));

    dxA = [dr1 dr dzc];
    clear dr dr1 drr dzc k

  case 3
    dzc = sparse(repmat(1,Nps*Nepoch,1));

    dxA = [dr1 dzc];
    clear dr1 dzc k

  otherwise
    error('You specified a wrong bowl model.');
end


% ------------------------------------------------------------------------
% Estimate parameters
% ------------------------------------------------------------------------

yi = C.*r1 + zc;
dy = y-yi;

switch defo_method
  case 1
    xi = [r1(1:Nps:Nps*Nepoch); r; xc; yc; zc];
  case 2
    xi = [r1(1:Nps:Nps*Nepoch); r; zc];
  case 3
    xi = [r1(1:Nps:Nps*Nepoch); zc];
end


Nx = dxA'*dxA;
rx = dxA'*dy;
ee = eig(Nx);
ee(1:10);
size_nn = size(null(full(Nx)))
%figure:imagesc(dxA(:,1:30));colorbar
%figure;imagesc(dxA(:,31:60));colorbar
%figure;imagesc(dxA(:,61));colorbar
%figure;imagesc(dxA(:,62));colorbar
%error('dfd')
dx = inv(Nx)*rx;
xhat = xi+dx;

r1 = kron(xhat(1:Nepoch),ones(Nps,1));

switch defo_method
  case 1
    r = xhat(Nepoch+1:2*Nepoch);
    xc = xhat(2*Nepoch+1);
    yc = xhat(2*Nepoch+2);
    zc = xhat(2*Nepoch+3);
  case 2
    r = xhat(Nepoch+1:2*Nepoch);
```

```
      zc = xhat(2*Nepoch+1);
    case 3
      zc = xhat(Nepoch+1);
  end

  q = dx'*Nx*dx
  v = v+1;

  if strcmp(wtest_flag,'yes')
    ddy = dxA*dx;
    de = dy-ddy;
    de = reshape(de,Nps,Nepoch);
    [dummy,index] = max(sum(abs(de),2));
    index
    y(index:Nps:Nepoch*Nps) = [];
    ps_az(index) = [];
    ps_r(index) = [];
    Nps = Nps - 1;

    xc = xc0;
    yc = yc0;
    zc = zc0;
    r = r0;
    r1 = kron(r10,ones(Nps,1));;

    v=1;
    q=inf;
    wtest_flag = 'no';
  end


  if q>q_old %no convergence, data snooping

    xc = xc0;
    yc = yc0;
    zc = zc0;
    r = r0;
    r1 = kron(r10,ones(Nps,1));;

    q = inf;
    wtest_flag = 'yes';
  end

end

r1 = r1(1:Nps:Nps*Nepoch);
r1 = m2ph*r1; %convert to phase


% ------------------------------------------------------------------------
% Estimate missing values by linear interpolation
% ------------------------------------------------------------------------

epoch_tot = 1:Nifgs;
epoch_missing = setdiff(epoch_tot,epoch);
if ~isempty(epoch_missing)
  r = interp1(epoch,r,epoch_tot,'linear','extrap');
  r1 = interp1(epoch,r1,epoch_tot,'linear','extrap');
end


% ------------------------------------------------------------------------
% Apply estimated deformation model
% ------------------------------------------------------------------------
```

```
psc_fid = fopen([project_id '_psc_sel' num2str(z) '.raw'],'r');
psc_data = fread(psc_fid,[2*Nifgs+7 Npsc(z)],'double')';

psc_azx = psc_data(:,5)*az_spacing; %[m]
psc_rx = psc_data(:,6)*r_spacing;
clear psc_data

psc_defo = repmat(NaN,Npsc(z),Nifgs);
for v = 1:Nifgs
  psc_defo(:,v) = -r1(v)*exp(-0.5*((psc_azx-xc).^2 + (yc-psc_rx).^2)./(r(v)^2))
+zc;
  %fig = fig+1;

%figure(fig);scatter(ps_data(:,2),ps_data(:,3),10,ps_data(:,Npar_max+6+v),'fille
d');colorbar
  %fig = fig+1;

%figure(fig);scatter(psc_data(:,3),psc_data(:,4),10,psc_defo(:,v)/m2ph,'filled')
;colorbar
  %display('pause')
  %pause
end

psc_defo_fid = fopen([project_id '_psc_defo_temp_sel' num2str(z) '.raw'],'w');
fwrite(psc_defo_fid,psc_defo','double');
fclose(psc_defo_fid);


switch ps_eval_method
  case 'psp'

    Npsp_buffer = floor(max_mem_buffer/(4*Nifgs*8));
    if Npsp_buffer>=Npsp(z)
      Npsp_buffer = Npsp(z);
      Npsp_rem_buffer = 0;
      Nbuffers = 1;
    else
      Nbuffers = floor(Npsp(z)/Npsp_buffer);
      Npsp_rem_buffer = rem(Npsp(z),Npsp_buffer);
      if (Npsp_rem_buffer > 0)
        Nbuffers = Nbuffers + 1;
      end
    end

    psp_fid = fopen([project_id '_psp_sel' num2str(z) '.raw'],'r');
    psp_az = repmat(NaN,Npsp(z),1);
    psp_r = repmat(NaN,Npsp(z),1);

    for v = 1:Nbuffers
      sl = (v-1)*Npsp_buffer+1;
      el = v*Npsp_buffer;
      if (v==Nbuffers)&&(Npsp_rem_buffer~=0)
        Npsp_buffer = Npsp_rem_buffer;
        el = Npsp(z);
      end

      psp_data = fread(psp_fid,[2*Nifgs+4 Npsp_buffer],'double')';

      psp_az(sl:el) = psp_data(:,3);
      psp_r(sl:el) = psp_data(:,4);
      clear psp_data
    end
    fclose(psp_fid);
```

```
    psp_azx = psp_az*az_spacing; % coordinates in meters
    psp_rx = psp_r*r_spacing;

    psp_defo = repmat(NaN,Npsp(z),Nifgs);
    for v = 1:Nifgs
      psp_defo(:,v) = -r1(v)*exp(-0.5*((psp_azx-xc).^2 + (yc-psp_rx).^2)./
(r(v)^2))+zc;
    end

    psp_defo_fid = fopen([project_id '_psp_defo_temp_sel' num2str(z)
'.raw'],'w');
    fwrite(psp_defo_fid,psp_defo','double');
    fclose(psp_defo_fid);

  case 'whole'

    % ---------------------------------------------------------------------
    % Write modeled deformation to file
    % ---------------------------------------------------------------------

    [az,r] = meshgrid((1:Nlines)*az_spacing,(1:Npixels)*r_spacing);
    for v = 1:Nifgs
      defo = -r1(v)*exp(-0.5*((az-xc).^2 + (yc-r).^2)./(r(v)^2))+zc;

      defo_fid = fopen([filenames_output(v,:) '_defo_sel' num2str(z)
'.raw'],'w');
      fwrite(defo_fid,defo','single');
      fclose(defo_fid);
    end

end
```